Lab Assignment 15.2: Backend API Development

Course: AI Assisted Coding (23CS002PC304)

Name:G.Varshith Reddy

Enrollment:2303a52287

Batch:37

Task Descriptor-1: (Setup Fast API Backend)

Task Description:

Install FastAPI and Uvicorn. Use AI to generate a basic FastAPI application with one endpoint.

Prompt:

> "Generate a basic FastAPI application setup with a single root endpoint that returns a welcome JSON message."

Code:

```python
from fastapi import FastAPI

# Initialize the FastAPI application
app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Output:

// Running the server and accessing http://127.0.0.1:8000/

```
{
    "message": "Welcome to AI-powered FastAPI"
}
```

Explanation:

Here we are setting up our main application by importing FastAPI and instantiating it. We define a simple root endpoint (/) that listens for GET requests and responds with a basic welcome message in JSON format.

Task Descriptor- 2: (API – Create and Read Items)

Task Description:

Implement endpoints to add and retrieve products. Use a Python list as temporary storage.

Prompt:

> "Write FastAPI endpoints to get all items from a temporary Python list and add a new item to the list using GET and POST HTTP methods."

Code:

```python
from fastapi import FastAPI

app = FastAPI()
items = [] # Temporary storage

@app.get("/items")
def get_items():
    return items

@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:

// GET /items
[]

// POST /items with body {"name":"Pen","price":20}

```
[
  {
    "name": "apple",
    "price": 1.5
  },
  {
    "name": "banana",
    "price": 0.75
  },
  {
    "name": "Pen",
    "price": 20
  }
]
```


Explanation:

We've introduced a Python list to serve as our temporary database for storing items. We created a GET route to retrieve all items from this list and a POST route that accepts new item data, appends it to our list, and returns a success message.

Task Descriptor- 3: (Update Existing Item)

Task Description:

Use AI to generate an endpoint to update an item using its index.

Prompt:

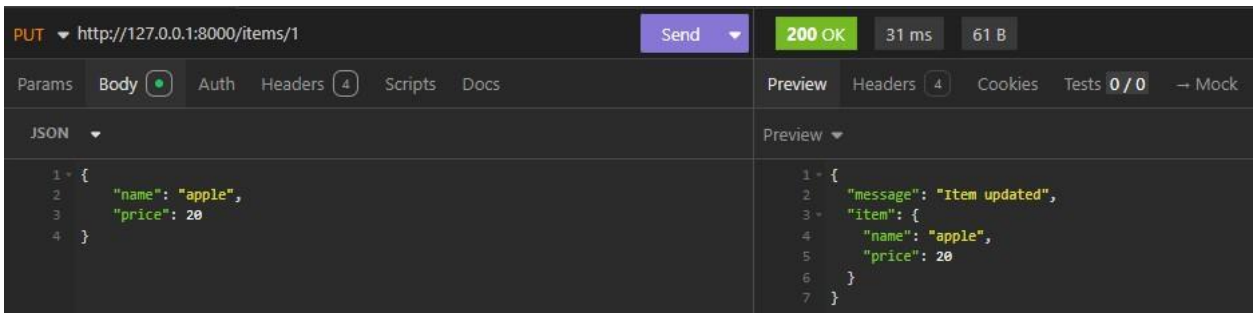
> "Create a PUT endpoint in FastAPI to update an existing item in a list based on its index. Make sure to handle the case where the index is out of bounds."

Code:

```python
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:

// PUT /items/0 with body {"name":"Pencil","price":10}
{"message": "Item updated", "item": {"name": "Pencil", "price": 10}}



Explanation:

This task adds a PUT endpoint allowing us to modify existing items by targeting their specific index in our list. It includes a simple validation check to ensure the index exists before updating the data, preventing potential out-of-range errors.

Task Descriptor- 4: (Delete Item)

Task Description:

Implement deletion of an item by index.

Prompt:

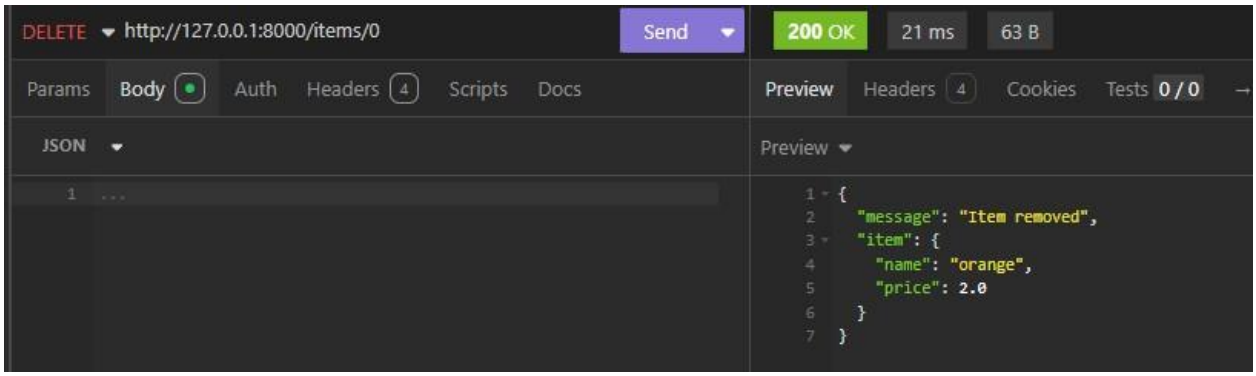
> "Create a DELETE endpoint in FastAPI to remove an item from a list by its index, returning the removed item and handling invalid indices."

Code:

```python
@app.delete("/items/{index}")
def delete_item(index: int):
    if index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output:

// DELETE /items/0
{"message": "Item removed", "item": {"name": "Pencil", "price": 10}}



Explanation:

We implemented a DELETE route that targets an item based on its URL path index. The function verifies the index is valid, removes the item from the list using the pop() method, and sends back the details of the removed item.

Task Descriptor -5: (Add Auto-Generated Documentation)

Task Description:

Use AI to add inline comments and docstrings for all endpoints. Optionally, integrate Swagger / Flask-RESTX to auto-generate API documentation.

Prompt:

> "Add detailed docstrings and inline comments to all my existing FastAPI CRUD endpoints so that the auto-generated Swagger documentation at /docs is fully descriptive."

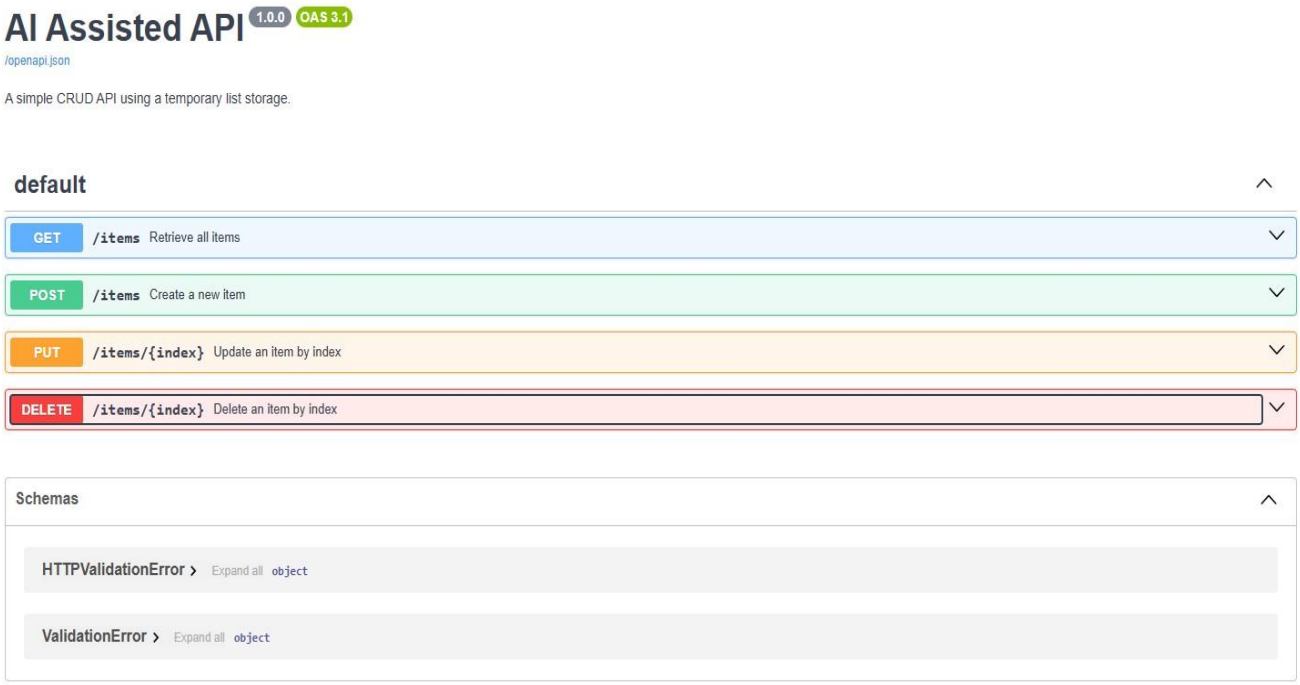
Code:

```python
from fastapi import FastAPI

app = FastAPI(
    title="AI Assisted API",
    description="A simple CRUD API using a temporary list storage.",
    version="1.0.0"
)

# Temporary in-memory storage for our items
items = []

@app.get("/items", summary="Retrieve all items")
def get_items():
    """
    Fetch all items currently stored in the temporary list.

    Returns:
        List[dict]: A list containing all the item dictionaries.
    """
    return items

@app.post("/items", summary="Create a new item")
def add_item(item: dict):
    """
    Add a new item to the store.

    - **item**: A dictionary containing the item's details (e.g., name and price).

    Returns:
        dict: A success message along with the created item.
    """
    items.append(item)
    return {"message": "Item created", "item": item}
```

```python
@app.put("/items/{index}", summary="Update an item by index")
def update_item(index: int, item: dict):
    """
    Update the details of an existing item using its list index.

    - **index**: The integer index of the item to update.
    - **item**: A dictionary containing the updated item details.

    Returns:
        dict: A success message and the updated item, or an error if not found.
    """
    if index >= len(items) or index < 0:
        return {"error": "Item not found"}

    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}", summary="Delete an item by index")
def delete_item(index: int):
    """
    Remove an item from the store using its list index.

    - **index**: The integer index of the item to be removed.

    Returns:
        dict: A success message and the removed item, or an error if not found.
    """
    if index >= len(items) or index < 0:
        return {"error": "Item not found"}

    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

Output:

// Accessing http://127.0.0.1:8000/docs Clear, interactive Swagger UI documentation is displayed.
Students can see endpoint descriptions, required path parameters (index), body payloads (item dictionary), and test the API directly from the browser.



Explanation:

FastAPI automatically generates interactive Swagger documentation at the /docs endpoint based on our route definitions. By adding descriptive docstrings to our Python functions, we enrich this auto-generated UI, making it easier for developers to understand and test the API.